

山东大学\_\_ 计算机科学与技术 \_\_学院

大数据分析实践 课程实验报告

|                                                                                                          |                  |        |
|----------------------------------------------------------------------------------------------------------|------------------|--------|
| 学 号：<br>202300130059                                                                                     | 姓名： 林子洋          | 班级： 数据 |
| 实验题目： 电子表格实践 I                                                                                           |                  |        |
| 实验学时： 2                                                                                                  | 实验日期： 2025.10.14 |        |
| 实验要求： Add a new vis function based on the open source spreadsheet                                        |                  |        |
| 硬件环境：<br>计算机一台                                                                                           |                  |        |
| 软件环境：<br>编程语言： Python 3.9<br>开发工具： Jupyter Notebook<br>核心库： Pandas、NumPy                                 |                  |        |
| 实验步骤与内容：<br>环境准备： 在 VS Code 中创建 HTML 文件，引入 x-spreadsheet（表格处理）和 d3（可视化）库。<br>页面结构设计： 添加表格容器、复选框（控制可视化开关） |                  |        |

和可视化结果容器。

表格初始化：配置 `x-spreadsheet`，设置初始数据（如不同年份的专业人数）。

可视化功能实现：

定义 `update` 函数，通过复选框状态触发，读取表格数据并转换格式。

使用 `d3` 绘制分组条形图，展示表格中的数据（如不同年份、不同专业的人数对比）。

事件绑定：将 `update` 函数绑定到表格编辑事件和复选框状态变化事件，实现动态更新。

实验代码如下：

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <link    rel="stylesheet"    href="https://unpkg.com/x-data-spreadsheet@1.1.5/dist/xspreadsheet.css" />
```

```
    <script                                src="https://unpkg.com/x-data-spreadsheet@1.1.5/dist/xspreadsheet.js"></script>
```

```
    <script                                src="https://unpkg.com/x-data-spreadsheet@1.1.9/dist/locale/zh-cn.js"></script>
```

```
    <script src="https://d3js.org/d3.v6.js"></script>
```

```
    <style>
```

```
#xspreadsheet { width: 400px; height: 500px; }
#my_dataviz { width: 1000px; height: 900px; }
.tick text { font-size: 12px; }
</style>
</head>
<body>
  <div id="xspreadsheet">
    <input      type="checkbox"      class="checkbox"
value="barchart" />
    <label>显示条形图</label>
  </div>
  <div id="my_dataviz"></div>

  <script>
    // 初始化表格（中文本地化）
    x_spreadsheet.locale("zh-cn");
    const xs = x_spreadsheet("#xspreadsheet", {
      mode: 'edit',
      row: { len: 15, height: 25 },
      col: { len: 8, width: 100 }
    });
```

```

// 设置初始数据（年份、计算机、法学人数）
xs.cellText(0, 1, " 计 算 机 ").cellText(0, 2, " 法 学
").reRender();

xs.cellText(1, 0, "2017").cellText(1, 1, "23").cellText(1,
2, "15").reRender();

xs.cellText(2, 0, "2018").cellText(2, 1, "36").cellText(2,
2, "26").reRender();

xs.cellText(3, 0, "2019").cellText(3, 1, "23").cellText(3,
2, "33").reRender();

xs.cellText(4, 0, "2020").cellText(4, 1, "22").cellText(4,
2, "10").reRender();


// 颜色映射函数
function getColor(idx) {
    const palette = ['#5ab1ef', '#ffb980', '#d87a80',
'#2ec7c9'];

    return palette[idx % palette.length];
}


// 可视化更新函数
function update() {

    const isChecked =

```

```
d3.select('.checkbox').property("checked");

    if (!isChecked) {
        d3.selectAll('svg').remove(); // 取消勾选时
清除图表

        return;
    }

    // 读取表格数据
    let yTitle = [], xTitle = [], data1 = [];
    // 获取行标题（年份）
    for (let i = 1; i < 20; i++) {
        const cell = xs.cell(i, 0);
        if (!cell || !cell.text) break;
        yTitle.push(cell.text);
    }
    // 获取列标题（专业）
    for (let i = 1; i < 20; i++) {
        const cell = xs.cell(0, i);
        if (!cell || !cell.text) break;
        xTitle.push(cell.text);
    }
    // 获取数值数据
```

```
        for (let i = 1; i < yTitle.length + 1; i++) {
            const row = [];
            for (let j = 1; j < xTitle.length + 1; j++) {
                const cell = xs.cell(i, j);
                if (!cell || !cell.text || isNaN(+cell.text))
return; // 校验数据格式
                row.push(+cell.text);
            }
            data1.push(row);
        }

// 处理数据格式为 d3 所需
const maxValue = Math.max(...data1.flat());
const data = yTitle.map((group, i) => {
    const obj = { group };
    xTitle.forEach((key, j) => obj[key] =
data1[i][j]);

    return obj;
});

// 绘制分组条形图
const margin = { top: 40, right: 30, bottom: 40, left:
```

```
50 };

    const width = 600 - margin.left - margin.right;
    const height = 500 - margin.top - margin.bottom;

    d3.selectAll('svg').remove(); // 清除旧图表
    const svg = d3.select("#my_dataviz")
        .append("svg")
        .attr("width", width + margin.left +
margin.right + 100)
        .attr("height", height + margin.top +
margin.bottom)
        .append("g")
        .attr("transform",
`translate(${margin.left},${margin.top})`);

    // x 轴（分组）
    const x = d3.scaleBand().domain(yTitle).range([0,
width]).padding(0.2);
    svg.append("g")
        .attr("transform", `translate(0, ${height})`)
        .call(d3.axisBottom(x).tickSizeOuter(0));
```

```

        // Y 轴（数值）
        const y = d3.scaleLinear().domain([0,
maxValue]).range([height, 0]).nice();
        svg.append("g").call(d3.axisLeft(y));

        // 子组 X 轴（专业）
        const xSubgroup =
d3.scaleBand().domain(xTitle).range([0,
x.bandwidth()]).padding(0.05);
        const color =
d3.scaleOrdinal().domain(xTitle).range(xTitle.map((_, i) =>
getColor(i)));

        // 绘制条形
        svg.append("g")
            .selectAll("g")
            .data(data)
            .join("g")
            .attr("transform", d =>
`translate(${x(d.group)}, 0)`)
            .selectAll("rect")
            .data(d => xTitle.map(key => ({ key, value:

```



```
d[key] })))
```

```
    .join("rect")  
    .attr("x", d => xSubgroup(d.key))  
    .attr("y", d => y(d.value))  
    .attr("width", xSubgroup.bandwidth())  
    .attr("height", d => height - y(d.value))  
    .attr("fill", d => color(d.key));
```

```
// 添加数值标签
```

```
svg.selectAll("g.bar")  
    .data(data)  
    .join("g")  
    .attr("transform", d =>  
`translate(${x(d.group)}, 0)`)  
    .selectAll("text")  
    .data(d => xTitle.map(key => ({ key, value:  
d[key] })))  
    .join("text")  
    .attr("x", d => xSubgroup(d.key) +  
xSubgroup.bandwidth() / 2)  
    .attr("y", d => y(d.value) - 5)  
    .text(d => d.value)
```

```
.attr("text-anchor", "middle");

// 添加图例
const legend = svg.selectAll(".legend")
  .data(xTitle)
  .join("g")
  .attr("class", "legend")
  .attr("transform", (d, i) => `translate(${width
+ 20}, ${i * 20})`);

legend.append("rect")
  .attr("width", 15)
  .attr("height", 15)
  .attr("fill", d => color(d));

legend.append("text")
  .attr("x", 20)
  .attr("y", 12)
  .text(d => d);
}

// 绑定事件（表格编辑和复选框变化时更新可视化）
```

```
        xs.on('cell-edited', update);
        d3.selectAll(".checkbox").on("change", update);
    </script>
</body>
</html>
```

结论分析与体会：

结论分析

功能实现：成功在 **x-spreadsheet** 基础上扩展了可视化功能，通过复选框控制分组条形图的显示 / 隐藏，表格数据修改时图表实时更新，实现了 “编辑 - 可视化” 联动。

数据流转：通过 **update** 函数完成表格数据的读取（行 / 列标题、数值）、格式校验（确保数值有效）和转换（适配 **d3** 数据格式），为可视化提供可靠输入。

可视化效果：使用 **d3** 绘制的分组条形图清晰展示了多维数据（如不同年份、不同专业的人数对比），并通过颜色编码和

图例增强可读性。

## 实验体会

库的协同性：**x-spreadsheet** 提供灵活的表格编辑能力，**d3** 专注于数据可视化，二者结合可快速构建 “数据录入 - 分析 - 可视化” 闭环，降低开发成本。

事件驱动设计：通过绑定表格编辑事件和复选框状态变化事件，实现了数据与可视化的动态同步，提升了交互体验，这是前端数据应用的核心设计思路。

数据校验重要性：实验中对表格数据进行格式校验（如判断是否为数值），避免了无效数据导致的可视化错误，体现了数据预处理在前端应用中的必要性。

可扩展性：此框架可进一步扩展（如支持折线图、饼图，或增加数据筛选功能），为复杂表格可视化场景提供了基础。

通过本次实验，理解了开源库二次开发的思路：基于现有工具的核心能力，通过事件绑定和数据流转扩展新功能，平衡开发效率与定制需求。

